

CSE305 : N-Body Simulation Project

École Polytechnique, Academic year 2024/2025

1	Introduction	2
1.1	Project Overview	2
1.2	Codebase and Experimental Setup	2
2	Creating a Sequential N-Body Simulation	2
2.1	Defining the Force Computing Algorithm	2
2.2	Creating a Universe of Astronomical Bodies	3
2.3	Visualization with Magick++	3
3	Parallelizing our Simulation	4
3.1	Parallelizing Force Computation	4
3.2	Implementing Dynamic Load Balancing	5
3.3	Parallelizing Position Updating	5
3.4	Parallelizing I/O to Prevent Bottlenecks	5
4	Implementing the Barnes-Hut Algorithm	6
4.1	Algorithm Intuition	6
4.2	Practical Implementation	7
4.3	Parallelization	7
4.4	Maximizing Accuracy-Performance Trade-off	7
5	GPU-Accelerated Force Computation with CUDA	8
5.1	Recreating a Parallelized Simulation in CUDA	8
5.2	Dynamic Load Balancing	8
5.3	Accuracy Benchmarking	9
6	Conclusion	9

[1] Josh Barnes and Piet Hut. A hierarchical $O(n \log n)$ force-calculation algorithm. nature, 324(6096), 1986.

1 Introduction

This report presents the work of a concurrent programming project for the CSE305 course at the Bachelor of Science of Ecole Polytechnique. Its goal is to implement a gravity simulation capable of calculating the trajectories of tens of thousands of astronomical bodies by considering all pairwise gravitational interactions (whose number grows quadratically relative to the number of bodies). This project explores how to implement and significantly optimize this simulation using parallel computing techniques.

1.1 Project Overview

This project is divided into multiple sections which we will cover throughout this report

1. **The straightforward implementation:** To start, we design a sequential N-Body simulation with its own renderer written with the *Magick++* library. To do so, we define pairwise interactions between bodies using Newton's law of universal gravitation and implement an algorithm to compute the resulting force applied on each body and update positions accordingly. Then, we use *Magick++* to create a renderer for our N-Body simulation.
2. **Parallelization:** To improve performance, we define a parallel algorithm with dynamic load balancing for force computation and position updating. We also parallelize the creation of images to avoid creating an I/O bottleneck by creating images and computing subsequent positions in parallel.
3. **Barnes-Hut Algorithm:** To further accelerate force computation, we implement the Barnes-Hut algorithm, allowing us to go from $O(n^2)$ to $O(n \log(n))$ performance.
4. **CUDA Implementation:** Lastly, we provide a CUDA implementation of our algorithm which takes advantage of the numerous GPU cores to perform force computation. This allows us to reach huge speed-ups and run algorithms with 100 thousand bodies over a month (with day-steps) in about 30 seconds.

1.2 Codebase and Experimental Setup

All relevant code for this project is available on my [GitHub repository](#). The code on the `main` branch was run on a Macbook Pro M1 (2021) with 10 CPU cores (2 efficiency cores, 8 performance cores). The code on the `cuda` branch was run on a Linux Machine (coucou.polytechnique.fr) with an x86_64 architecture, an Intel(R) Xeon(R) W-1270P with 16 available threads, and an A5000 NVIDIA GPU.

2 Creating a Sequential N-Body Simulation

To start this project, we define an implementation of an N-Body Simulation in C++ capable of computing forces sequentially, update the positions of bodies in a universe, and render this simulation in a user-friendly manner.

2.1 Defining the Force Computing Algorithm

Newton's law of universal gravitation states that two astronomical bodies b_1 and b_2 continuously attract each other. b_1 applies a force (in Newtons) on b_2 in the direction $b_2 \rightarrow b_1$ and b_2 applies a force on b_1 in the direction $b_1 \rightarrow b_2$. Given respective masses m_1 and m_2 and distance from each other d , the magnitude of these forces is:

$$F_{b_1, b_2} = F_{b_2, b_1} = G \frac{m_1 \times m_2}{d^2}$$

where $G = 6.67430 \times 10^{-11} \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-2}$ is the universal gravitational constant.

In our simulation, we need to compute every pairwise force between bodies. Given N bodies, this is like filling up the matrix:

$$\begin{bmatrix} \vec{F}_{b_1,b_1} & \vec{F}_{b_1,b_2} & \vec{F}_{b_1,b_3} & \vec{F}_{b_1,b_4} & \cdots & \vec{F}_{b_1,b_N} \\ \vec{F}_{b_2,b_1} & \vec{F}_{b_2,b_2} & \vec{F}_{b_2,b_3} & \vec{F}_{b_2,b_4} & \cdots & \vec{F}_{b_2,b_N} \\ \vec{F}_{b_3,b_1} & \vec{F}_{b_3,b_2} & \vec{F}_{b_3,b_3} & \vec{F}_{b_3,b_4} & \cdots & \vec{F}_{b_3,b_N} \\ \vec{F}_{b_4,b_1} & \vec{F}_{b_4,b_2} & \vec{F}_{b_4,b_3} & \vec{F}_{b_4,b_4} & \cdots & \vec{F}_{b_4,b_N} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \vec{F}_{b_N,b_1} & \vec{F}_{b_N,b_2} & \vec{F}_{b_N,b_3} & \vec{F}_{b_N,b_4} & \cdots & \vec{F}_{b_N,b_N} \end{bmatrix}$$

The most basic algorithm could therefore be to run two nested `for` loops that fill up the matrix entry by entry. However, from the skew-symmetry (symmetry with a -1 factor) of gravitational forces, we can already halve the number of required computations by only computing the upper triangle matrix and filling the rest by skew-symmetry.

```
for (size_t i = 0; i < numBodies; ++i) {
    for (size_t j = i + 1; j < numBodies; ++j) {
        Vector forceIJ = computePairwiseForce(bodies[i], bodies[j]);
        forces[i] += forceIJ;
        forces[j] += -forceIJ;
    }
}
```

Figure 1: Sequential Force Computation Algorithm.

2.2 Creating a Universe of Astronomical Bodies

Beyond force computation, to obtain the trajectories of a large number of bodies in space, one first needs to define how the universe and bodies will work. Bodies are thus objects defined by their mass, their position, their velocity, and their radius. Universes are also set as objects defined by the bodies they contain. Our program runs in a main simulation loop which will update the positions of each body within a given universe at each step for a given amount of iterations.

To update positions, our sequential program first computes the forces using the algorithm defined in Section 2.1. Then, we use Newton's second law of motion to state that $\vec{F} = m\vec{a}$. Since we now know the force applied on each body, we can update positions by using acceleration as follows:

```
for (size_t i = 0; i < bodies.size(); ++i) {
    Body &body = bodies[i];
    const Vector acceleration = forces[i] / body.getMass();
    const Vector velocity = body.getVelocity() + timestep * acceleration;
    body.setVelocity(velocity);
    const Vector position = body.getPosition() + timestep * velocity;
    body.setPosition(position);
}
```

Figure 2: Sequential Position Updating Algorithm.

2.3 Visualization with Magick++

Using the *Magick++* library, we can then create a renderer object which will create an image at every iteration of the loop displaying the current body positions and that will concatenate every frame in a GIF at 24 FPS at the

end of the simulation. This allows us to get impressive visual results and verify the correctness of our program.

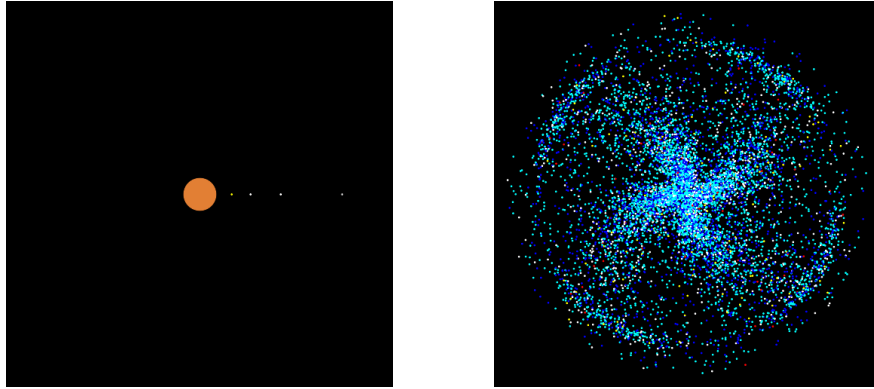


Figure 3: Rendering of Jupyter and its four biggest moons (left) and of the Milky Way (right)

Given the heavy costs of rendering due to I/O bottlenecks, we nonetheless set rendering off by default to get benchmarks that are not dependent on rendering speeds that are not the core focus of our project.

3 Parallelizing our Simulation

Despite the fact that our simulation works pretty well and yields very good visual results, its execution time is rather high. This is due to manipulating complex objects, the I/O bottleneck caused by rendering, and the complexity of our force computation algorithm which is quadratic ($O(n^2)$) relative to the amount of bodies in our universe. The goal of this section is thus to compensate for the program complexity with parallel computing techniques.

3.1 Parallelizing Force Computation

When running our sequential implementation, we notice that most of the program spends time computing forces. For a simulation with 5 thousand bodies that lasts one month (with day-steps), our program executes in 4.45 seconds and that 99.9% of that time is spent calculating all of the pairwise interactions.

To significantly speed up our simulation, we can create a new algorithm that will concurrently calculate all of the pairwise interactions. Looking back at our algorithm in Figure 2, the most basic idea is thus to divide the outer loop into chunks that each thread computes in parallel. Going back to the upper triangular force matrix, this would mean that thread 1 computes forces for rows 0 to `chunk_size`, thread 2 from row `chunk_size` to $2 \times \text{chunk_size}$, etc (while filling the rest via skew-symmetry).

This allows for significant improvements, as can be seen in that table:

Number of Threads	1	2	3	4	5	6	7	8	9	10
Execution Time (s)	123	97.1	76.6	61.9	53.1	46.7	42.3	38.0	35.3	33.3
Speedup Factor	1.00	1.27	1.61	1.99	2.32	2.63	2.91	3.24	3.49	3.69

Table 1: Execution times and speedup relative to 1 thread.

We also note that in comparison with the sequential implementation, we have a 0.00% divergence in terms of body positions, confirming the correctness of our algorithm. However, despite these speed-ups, we are not at all close to optimal performance. Having only a $3.70\times$ speedup with $10\times$ more threads should raise a flag. Indeed, our simulation suffers from a major load balancing flaw.

3.2 Implementing Dynamic Load Balancing

With the given algorithm, the work done by each thread is heavily unbalanced. Each thread is in charge of computing forces for a set of consecutive rows from the upper triangular matrix. However, the height of each row varies significantly. The first few threads will compute $j - i$ forces with i close to 0 while the last few threads will compute $j - i$ forces with i close to j . Concretely, with 10 threads and one thousand bodies, the first thread will compute 95 thousand pairwise forces (19% of the work instead of 10%) while the last thread will only compute 5 thousand forces (1% of the work instead of 10%). Since we wait for all threads to terminate before continuing, this is highly inefficient.

To solve that issue, one idea would be to distribute the rows to compute differently. You could have that each thread computes forces for all multiples of `numThreads`, with a `threadId` offset. With two threads, thread 1 would therefore take all of the odd rows and thread 2 the even rows. However, with a growing number of threads (ex 28 on the best lab machine), the unbalancing problem starts to appear again (in a much weaker form). To even further optimize our algorithm, we can thus define an atomic counter which will point to the next available row. Each thread will thus compute the next available row, enabling perfect thread balancing (and countering the effect of anomalies). We also experimentally note that despite slight contention in the atomic counter, the benefits this methods brings outweigh this delay. In the end, we see that runtime speeds up close to linearly relative to the amount of threads. This is much more convincing in comparison to Section 3.1:

Number of Threads	1	2	3	4	5	6	7	8	9	10
Execution Time (s)	110	57.2	38.7	28.6	22.8	19.2	16.7	14.4	13.8	13.5
Speedup	1.00	1.92	2.84	3.85	4.82	5.73	6.59	7.64	7.97	8.15

Table 2: Execution times and speedup relative to 1 thread.

3.3 Parallelizing Position Updating

At each simulation loop, our algorithm goes through each body and updates its position. This algorithm is not inherently slow as its complexity grows linearly relative to the number of bodies (unlike the forces computation algorithm which grows quadratically). For small amounts of bodies, the overhead costs of parallelization outweigh its benefits. However, for a large number of bodies (experimentally, ~ 1 thousand is a good cutoff), parallelizing this algorithm is beneficial. Above this threshold, we divide the set of bodies into chunks that are updated in parallel. Due to the nature of position updates, we also note that load balancing is not an issue.

3.4 Parallelizing I/O to Prevent Bottlenecks

When rendering is activated, the I/O bottleneck it creates becomes the dominant component of our program. Initially, our simulation loops works sequentially as follows: 1. Compute the forces acting on each body, 2. Update the position of each body, 3. Create an image with updated positions.

In practice, with 5 thousand bodies over 10 days (with day-steps), our 10-thread parallel algorithm executes in 16.3 seconds, with 73.5% of that time devoted to image creation and 26.3% to force computation (position updating is negligible). This I/O bottlenecking can also be optimized.

To do so, we dedicate one of our 10 threads to rendering and change our simulation loop. After computing the forces acting on each body and updating their positions, our new algorithm starts over the loop while creating the image in parallel. Only when it reaches the next image creation task, it waits for the previous image creation to complete. We thus don't have to wait for image creation to complete to start the next loop.

With the same conditions as before, our 10-thread parallel algorithm now executes in 8.83 seconds with 98.7% of that time devoted to image creation and 55.7% of that time devoted to force computation. This repartition means that computing forces is constantly faster than creating images. While image creation happens continuously, the rest of the loop spends about half of the time computing forces and half waiting for image creation to terminate (as position updating is again negligible).

Here, we also note that given the heavy computations of image creation, dedicating one thread to image creation is always worth it. This is experimentally verified with the following results (as speedup with two threads > 1).

Number of Threads	Execution time (s)	Speedup	Proportion of time creating images (%)
1	32.3	1.00	26.2
2	30.6	1.06	27.9
4	25.6	1.26	34.1
6	18.9	1.71	45.8
8	12.4	2.6	75.0
10	8.83	3.66	98.7

Table 3: Rendering Performance for different numbers of threads.

This table also shows that 10 threads are necessary for force computation to become faster than image creation.

4 Implementing the Barnes-Hut Algorithm

Despite parallelization, our force computation algorithm is slow due to its quadratic complexity relative to the number of bodies. To mitigate that issue, our project also studies alternative methods such as the Barnes-Hut algorithm which enables force computation to go from $O(n^2)$ complexity to $O(n \log n)$.

4.1 Algorithm Intuition

The Barnes-Hut algorithm starts by creating a tree whose head is the whole space. Each node then has four children, one for each quadrant in space. We then recursively divide the space into quadrants such that leaves have at most one body. This gives a partition of the space as follows:

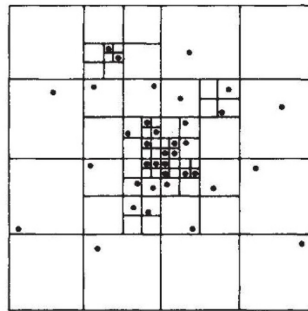


Figure 4: Set of bodies and induced subdivision of the space (from original paper).

Then, to compute the force on a given body, we start at the root of the tree and decide whether a node is far enough to be treated as a single mass or if we need to look at its children. If the width covered by the node divided by its distance with the target body is below some threshold θ , we consider all bodies under that node as a single body located at its center of mass. Otherwise, we look at the node's children which divide the node's coverage in four and repeat the same decision process. This algorithm thus allows nearby bodies to be handled individually and far away clusters to be considered as one bigger body.

4.2 Practical Implementation

In practice, we start by defining a Node object that contains the bounds of the space it covers, its center of mass and total mass, a body (if its a leaf), and pointers to four children nodes. We then define a Barnes-Hut object which contains the root of the tree, a method to build the tree, methods to compute the forces acting on each body, and a theta approximation parameter.

Using this implementation, with a high precision theta parameter (more about divergence and the choice of theta in Section 4.4) we are able to get a $7.97\times$ speed-up over the sequential implementation while maintaining a $<1\%$ divergence metric for 5 thousand bodies.

4.3 Parallelization

The Barnes-Hut algorithm is also parallelizable. Instead of going through each body one-by-one to compute all of the forces acting on it, one idea is to divide the set of bodies into chunks and have each thread compute the forces acting on each body for a specific chunk. However, even though it doesn't seem like it at first glance, we also have to worry about dynamic load balancing for the Barnes-Hut algorithm. Depending on how the vector of bodies is constructed, some threads may end up working on isolated bodies while others handle clustered bodies. This creates an imbalance. The thread working on isolated bodies will approximate forces often and terminate quickly while the thread dealing with clustered bodies will perform many exact force calculations. To solve that issue, we can again use an atomic counter, enabling threads to always work on the next available body.

Given the same purely random simulation with 5 thousand bodies over a month with day-steps, we observe the significant benefits of the Barnes-Hut algorithm as well as the positive effect of dynamic load balancing:

Threads	Parallel Implementation (s)	Barnes-Hut (s)	Barnes-Hut with Load Balancing (s)
1	123	13.9 (8.88 $\times$)	13.9 (8.87 $\times$ & 1.00 $\times$)
2	97.1	7.12 (13.6 $\times$)	7.08 (13.7 $\times$ & 1.01 $\times$)
3	76.6	4.88 (15.7 $\times$)	4.86 (15.8 $\times$ & 1.00 $\times$)
4	61.9	3.93 (15.8 $\times$)	3.65 (16.9 $\times$ & 1.08 $\times$)
5	53.1	3.04 (17.5 $\times$)	2.95 (18.0 $\times$ & 1.03 $\times$)
6	46.7	2.51 (18.6 $\times$)	2.47 (19.0 $\times$ & 1.02 $\times$)
7	42.3	2.26 (18.7 $\times$)	2.15 (19.7 $\times$ & 1.05 $\times$)
8	38.0	1.97 (19.3 $\times$)	1.93 (19.7 $\times$ & 1.02 $\times$)
9	35.3	2.07 (17.1 $\times$)	1.86 (19.0 $\times$ & 1.11 $\times$)
10	33.3	2.07 (16.1 $\times$)	1.90 (17.5 $\times$ & 1.09 $\times$)

Table 4: Barnes-Hut algorithm speedup over parallel implementation (with $\theta = 0.20$)

4.4 Maximizing Accuracy-Performance Trade-off

In the Barnes-Hut algorithm, we set a θ precision parameter which allows us to define when our algorithm should approximate forces and when it should compute exact forces. The higher this θ , the more you approximate. However, approximating comes at a cost. Over time, the positions of bodies will diverge from their actual trajectory. Given the random simulation with 5 thousand bodies over 31 days with day-steps, the average divergence of the position of a body compared to what it should have been can be computed as a function of the θ parameter as follows:

θ parameter	0.05	0.10	0.20	0.25	0.5	1.0	1.5	2.0
Force Calc. (s)	9.35	5.20	2.27	1.68	0.82	0.40	0.32	0.18
Divergence (%)	0.08	0.50	0.74	1.17	2.22	11.47	10.52	8.98

Table 5: Average divergence as a function of θ for the uniformly random simulation

This divergence is quite high for 31 steps. This happens because our simulation uses a uniform random distribution of bodies, which leads to more frequent approximations and thus higher divergence. In contrast, simulations like the Milky Way that have clusters of bodies compute exact forces more often and consequently reduce divergence in exchange for a slightly higher force computation time.

θ parameter	0.05	0.10	0.20	0.25	0.5	0.75	1.0	1.5	2.0
Force Computation Time (s)	22.1	10.0	4.06	3.13	1.33	0.953	0.754	0.604	0.519
Avg. Divergence (%)	1.6e-5	3.9e-5	9.0e-4	9.2e-4	2.2e-2	1.6e-2	3.7e-2	2.7e-1	4.1e-1

Table 6: Average divergence as a function of θ for the Milky Way simulation

In any case, since we want to keep our divergence metric under 1% regardless of the configuration and that force computation $< 5s$ is still very satisfying for 10 thousand bodies, we consider worst case scenarios and choose $\theta = 0.20$ as our parameter for benchmarks.

5 GPU-Accelerated Force Computation with CUDA

The CUDA framework enables us to use NVIDIA GPUs (which have huge amount of threads available) to perform computations in extreme parallel. On the coucou.polytechnique.fr lab machine, you have access to a NVIDIA RTX A5000 with 48 blocks of 256 threads per block, enabling 12 288 concurrent threads. This large parallelization power can then be put into use in order to greatly accelerate force computations.

5.1 Recreating a Parallelized Simulation in CUDA

In order to benefit from the CUDA framework, we first have to recreate our parallel force computation algorithm in a `.cu` file. Following a similar approach to CSE305 lab session 3, we thus adapt our force computation functions to make them CUDA compatible, which requires more memory management and knowledge of CUDA documentation.

This implementation in CUDA of our base parallel algorithm (without load balancing) allows us to reach huge speedups. Our CUDA version indeed executes in 1.28 seconds, a $26.2\times$ improvement over our parallel algorithm (without load balancing) and with 10 threads.

As a side remark, note that it could theoretically possible to implement the Barnes-Hut algorithm in CUDA, but it would be dramatically more complicated due to the complexity of handling trees in CUDA and the limited capabilities of recursive algorithms in CUDA.

5.2 Dynamic Load Balancing

Just like in the standard parallel implementation, we also have to worry about load balancing when using CUDA. Currently, despite huge speedups, given one thousand bodies, the first thread performs $1000\times$ more work than the last thread (1000 computations vs. 1).

However, unlike in the CPU-based parallel algorithm, using an atomic counter would not be viable here. Indeed, even though 10 threads do not imply heavy contention, 12 288 threads do. Each thread would thus spend

most of its life waiting for other threads to update the counter. To counter that, one option would again be to have each thread compute rows for multiples of `numThreads` with a `threadId` offset. Nonetheless, as most of our simulations have at most 10 thousand bodies and that the maximum number of threads is 12 288, this would be useless. We therefore have to find another way to perform dynamic load balancing.

The optimal algorithm is therefore to allocate, within the upper triangular matrix, a matrix entry instead of a matrix row to each thread. Then, using the thread id, we can ensure that the load is perfectly shared (as the number of entries in the upper triangular matrix is $\frac{n(n-1)}{2} \gg 12,288$ in our large simulations). The only difficulty is thus to cover all possible (i, j) pairs in the upper triangular matrix. To do so, our algorithm flattens the set of indices and make each thread compute entries for multiples of `numThreads` with a `threadId` offset.

With this new load balancing, we are able to run our large random simulation (with 5 thousand bodies over a month using day-steps) in 0.660 seconds, which provides a $1.93\times$ improvement over the CUDA version without load balancing and a $20.4\times$ improvement over our parallel implementation (with load balancing).

5.3 Accuracy Benchmarking

Using GPUs to perform numerical computations tends to introduce small numerical error as they have a lower floating-point precision for operations and functions. As we get that, with our CUDA implementation, the average divergence per body after one month is only of order $10^{-10}\%$, we can thus ensure that our algorithm is viable and of high precision.

6 Conclusion

In conclusion, in our N-Body simulation project, we have developed a program capable of computing the trajectories of large number of bodies with a sequential algorithm, a parallel algorithm with dynamic load balancing, a parallel Barnes-Hut algorithm with dynamic load balancing, and CUDA ultra-parallel algorithm with dynamic load balancing.

The latter algorithm is the fastest and allows us to perform force computation extremely efficiently, mitigating the complexity of the $O(n^2)$ part of our program. Pushing our implementation to the limit, we are able to run the simulation over a month (with day-steps) for 100 thousand bodies in about 30 seconds with a divergence metric in the order of $10^{-10}\%$. This result is impressive and makes the results of our simulation very satisfying.

In addition to all of that, we have also coded a renderer based on the *Magick++* library to visualize our simulation and written scripts to test different algorithm parameters (such as θ for the Barnes-Hut Algorithm) and compare the speed and accuracy of our different implementations.